

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Deeksha	Reg. No.:	192424097
Section / Batch:	2024-C	Date:	17-8-2026

Code of Conduct

I _____ Deeksha(192424097)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Deeksha _____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.

However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input

Example customer statement:

"I reported the damage to the car after the accident."

The sentence can produce different interpretations depending on whether **"to the car"** is associated with *reported* or *damage*.

Tasks:

- a) Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.
- b) Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.
- c) Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.
- d) Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

2. Intelligent Railway Ticket Booking Assistant

A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.

The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases
- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries

Example passenger request:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

The system sometimes incorrectly associates **"with AC sleeper"** with the passengers instead of the ticket/class information.

Tasks:

- a) Analyze the possible syntactic interpretations of the given passenger request using CFG.
- b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.
- c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.
- d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved
- Obligations

- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.

Tasks:

- Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.
- Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.
- Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.
- Develop a step-by-step processing workflow showing how the system can extract:
 - Supplier
 - Action
 - Equipment
 - Company
 - Obligation
 - Deadline

and justify how the proposed approach improves automated contract analysis.

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter natural-language queries rather than selecting predefined filters.

The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements

Example user query:

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate "**for students**" with the touchscreen rather than the laptop.

Tasks:

- Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.
- Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.
- Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.
- Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.

The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users

Example spoken command:

"Change my flight to Mumbai tomorrow with two passengers."

Depending on the parse, "**with two passengers**" may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.

The passenger may also say:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore."

Tasks:

- a) Analyze the possible syntactic structures and ambiguities in the given voice command.
- b) Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.
- c) Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.
- d) Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

a) Structural Ambiguity Using CFG

The sentence is:

“I reported the damage to the car after the accident.”

A CFG can represent it as:

$S \rightarrow NP VP$

$NP \rightarrow \text{Pronoun}$

$VP \rightarrow V NP PP PP$

$NP \rightarrow \text{Det } N$

$PP \rightarrow P NP$

The sentence has PP attachment ambiguity.

Interpretation 1:

“to the car” describes the damage.

I reported [the damage [to the car]] after the accident.

Meaning: The car was damaged, and the claimant reported that damage.

Interpretation 2:

“to the car” can be attached to the reporting action.

I reported [the damage] [to the car] after the accident.

A basic CFG can generate both structures but cannot decide which meaning is correct.

b) Limitations of Basic CFG

Basic CFG has several limitations:

1. **Ambiguity:** It may generate multiple parse trees.
2. **PP attachment:** It cannot easily identify whether a PP modifies a noun or verb.
3. **Agreement:** It does not effectively represent subject–verb agreement.
4. **Long sentences:** Long customer descriptions can produce many possible parses.
5. **Incomplete input:** It performs poorly with conversational or incomplete statements.
6. **No probability:** It cannot rank different interpretations.
7. **Limited semantics:** It mainly handles syntax and does not directly identify claim-related meanings.

Therefore, CFG alone is not sufficient for a reliable insurance claim system.

c) Improved Parsing Approach

The proposed system combines **PCFG, Feature Structures, and Earley Parsing**.

Customer Statement



Preprocessing



Tokenization



Earley Parser



Possible Parse Trees



PCFG Ranking



Feature Structure Checking



Information Extraction



Claim Details

Earley Parsing:

Handles ambiguous, lengthy, and incomplete sentences efficiently.

PCFG:

Assigns probabilities to different parse structures and selects the most likely interpretation.

Feature Structures:

Store grammatical features such as:

Person = First

Number = Singular

Tense = Past

They help identify agreement errors and improve grammatical analysis.

For the given sentence, the extracted information can be:

Field	Extracted Information
Claimant	I
Action	Reported
Damaged Object	Car
Cause	Accident
Time	After the accident

d) Justification

The proposed architecture improves the insurance claim system in the following ways:

- **Accuracy:** PCFG helps select the most probable interpretation.
- **Ambiguity Resolution:** Multiple parse structures can be compared.
- **Agreement Checking:** Feature Structures identify grammatical inconsistencies.
- **Efficiency:** Earley Parsing handles complex structures effectively.
- **Scalability:** The system can process a large number of customer claims.
- **Reliability:** Correct extraction of claim information reduces processing errors.
- **Conversational Support:** Incomplete and informal customer statements can be handled better.

Conclusion

A combination of CFG, PCFG, Feature Structures, and Earley Parsing provides a more accurate and reliable solution than basic CFG alone. PCFG resolves ambiguity, Feature Structures handle grammatical constraints, and Earley Parsing supports complex and incomplete inputs. Therefore, the proposed architecture is suitable for automated and scalable insurance claim processing.

2. Intelligent Railway Ticket Booking Assistant

a) Syntactic Interpretations Using CFG

The passenger request is:

“Book two tickets from Chennai to Delhi for my parents with AC sleeper.”

A simple CFG can be represented as:

$S \rightarrow VP$

$VP \rightarrow V NP PP PP PP PP$

$NP \rightarrow Num N$

$PP \rightarrow P NP$

The intended interpretation is:

Book

|— two tickets

|— from Chennai

|— to Delhi

|— for my parents

|— with AC sleeper

Here, “with AC sleeper” describes the ticket/class.

Another possible interpretation is:

Book

|— two tickets

|— from Chennai

|— to Delhi

└─ for my parents

└─ with AC sleeper

Here, “**with AC sleeper**” is incorrectly attached to “**my parents.**”

Thus, the sentence has prepositional phrase attachment ambiguity. CFG can generate both structures but cannot determine the intended meaning by itself.

b) Problems with Top-Down Parsing

Top-down parsing starts from the start symbol and tries to construct the sentence by predicting grammar rules.

For example:

S

↓

VP

↓

V NP PP

↓

V NP PP PP

↓

V NP PP PP PP

When the selected structure does not match the input, the parser must **backtrack** and try another possibility.

This creates several problems:

1. **Excessive backtracking:** Many possible PP attachments may have to be tested.
2. **Slow processing:** Complex passenger requests require more parsing steps.
3. **Ambiguity:** It may not know which phrase a PP modifies.
4. **Incomplete requests:** Inputs such as “*Book two tickets from Chennai...*” may be difficult to process.
5. **Long-distance dependencies:** Words or phrases that are related but far apart can complicate parsing.
6. **Real-time limitation:** Repeated backtracking increases response time.

c) Earley Parsing

Earley Parsing is more suitable for handling complex and ambiguous passenger requests.

For the given sentence, it can maintain multiple possible parsing states:

Book two tickets from Chennai to Delhi

↓

Partial Parse

↓

for my parents



with AC sleeper



Complete Parse

Earley Parsing provides the following advantages:

- **Handles ambiguity:** Maintains different possible parse structures.
- **Handles incomplete input:** Can process a request before the entire sentence is completed.
- **Reduces unnecessary backtracking:** Previous parsing states are stored and reused.
- **Handles complex structures:** Suitable for sentences with multiple phrases.
- **Supports conversational input:** Useful when passengers provide information gradually.

The system can then use probability or semantic information to identify:

Tickets = 2

Source = Chennai

Destination = Delhi

Passengers = Parents

Class = AC Sleeper

Thus, “**with AC sleeper**” is correctly interpreted as the ticket/class information.

d) Comparison of Parsing Methods

Factor	Top-Down Parsing	CFG-Based Parsing	Earley Parsing
Ambiguity handling	Poor	Generates multiple possibilities	Good
Backtracking	High	Depends on implementation	Reduced
Efficiency	Low for complex input	Moderate	Good
Partial input	Poor	Limited	Good
Long-distance dependencies	Difficult	Moderate	Better
Complex queries	Slow	Moderate	Efficient
Conversational input	Limited	Limited	Suitable
Real-time suitability	Low	Moderate	High

Conclusion

Top-down parsing is simple but can suffer from excessive backtracking and slow processing when passenger requests are complex or ambiguous. CFG provides the basic grammatical structure but cannot resolve ambiguity by itself. Earley Parsing is more suitable because it can handle ambiguity, incomplete input, and multiple parse structures efficiently.

Therefore, an Earley-based parsing system is a better choice for a real-time railway ticket booking assistant because it can provide faster, more accurate, and more reliable interpretation of passenger requests.

3. AI-Based Legal Contract Analysis System

a) Syntactic Structure Using CFG

The given sentence is:

“The supplier who delivered the equipment to the company must replace the defective components within thirty days.”

A simple CFG representation is:

$S \rightarrow NP VP$

$NP \rightarrow Det N RelClause$

$RelClause \rightarrow RelPronoun VP$

$VP \rightarrow V NP PP$

$VP \rightarrow Modal V NP PP$

$NP \rightarrow Det N$

$PP \rightarrow P NP$

The sentence can be structured as:

S
├─ NP
│ ├── The supplier
│ └─ Relative Clause
│ ├── who
│ └─ delivered
│ ├── the equipment
│ └─ to the company
└─ VP
 ├── must replace

- |— the defective components
- |— within thirty days

The intended relationships are:

Supplier → delivered → Equipment

Equipment → to → Company

Supplier → must replace → Defective Components

Deadline → Thirty days

The main ambiguity occurs in the phrase **“to the company.”** A basic parser may incorrectly associate the company with the action of delivery or identify it as the responsible entity.

b) Limitations of Basic CFG

Basic CFG is insufficient for complex legal sentences because:

1. Relative clauses:

The phrase *“who delivered the equipment”* introduces a relationship between the supplier and the delivery action.

2. Nested structures:

Legal sentences often contain clauses inside other clauses, making parsing more difficult.

3. PP attachment ambiguity:

“to the company” may be attached incorrectly.

4. No probability:

CFG cannot determine which of multiple valid parse trees is most likely.

5. Weak feature handling:

It does not effectively represent grammatical features such as number, person, tense, and case.

6. Long sentences:

Complex legal sentences can generate many possible parse structures.

7. Limited semantic interpretation:

CFG identifies grammatical structure but does not directly identify who performs an action or who receives it.

Therefore, basic CFG alone may produce incorrect interpretations of legal obligations and responsibilities.

c) Improved NLP Architecture

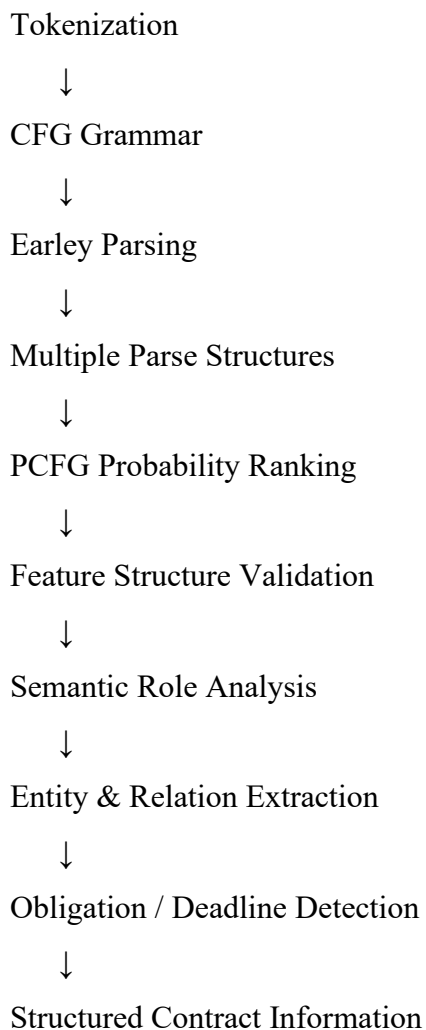
The proposed system integrates CFG, PCFG, Feature Structures, and Earley Parsing.

Legal Contract

↓

Text Preprocessing

↓



CFG

CFG provides the basic grammatical rules needed to identify nouns, verbs, clauses, and phrases.

Earley Parsing

Earley Parsing efficiently handles **ambiguous, long, and nested sentences** and maintains possible parsing states.

PCFG

PCFG assigns probabilities to possible parse structures and selects the most likely interpretation.

For example:

Supplier → delivered → Equipment

Equipment → received by → Company

This is ranked as the appropriate interpretation rather than treating all structures equally.

Feature Structures

Feature Structures store grammatical information such as:

Number = Singular

Tense = Past

Person = Third

They help maintain grammatical consistency during parsing.

d) Step-by-Step Processing Workflow

Step 1 – Input Processing

The contract sentence is provided to the NLP system.

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

Step 2 – Tokenization

The sentence is divided into tokens:

The | supplier | who | delivered | the | equipment |
to | the | company | must | replace | the | defective |
components | within | thirty | days

Step 3 – Syntactic Parsing

Earley Parsing identifies the main sentence and relative clause:

Main Clause:

The supplier must replace the defective components.

Relative Clause:

who delivered the equipment to the company

Step 4 – Ambiguity Resolution

PCFG ranks possible interpretations and selects:

Supplier → delivered → Equipment

Equipment → to → Company

This prevents the system from incorrectly treating the company as the delivery agent.

Step 5 – Feature Validation

Feature Structures check grammatical information such as:

Supplier:

Number = Singular

Person = Third

Delivered:

Tense = Past

Step 6 – Entity Extraction

The system identifies:

Entity	Extracted Information
Supplier	The supplier
Equipment	The equipment
Company	The company
Components	Defective components

Step 7 – Action Extraction

The actions are:

Action 1 = Delivered

Agent = Supplier

Object = Equipment

Action 2 = Replace

Agent = Supplier

Object = Defective Components

Step 8 – Obligation Detection

The modal word “**must**” indicates a mandatory obligation.

Responsible Party = Supplier

Obligation = Replace defective components

Step 9 – Deadline Detection

The phrase:

“**within thirty days**”

is identified as the deadline.

Deadline = 30 days

Final Extracted Information

Required Information	Result
Supplier	The supplier
Action	Replace
Equipment	The equipment
Company	The company
Obligation	Must replace defective components
Deadline	Within thirty days

Justification

The proposed architecture improves legal contract analysis because:

- **CFG** provides the basic grammatical structure.
- **Earley Parsing** handles long, nested, and ambiguous sentences.
- **PCFG** selects the most probable interpretation.
- **Feature Structures** maintain grammatical constraints.
- **Semantic analysis** identifies relationships between entities and actions.
- **Information extraction** converts legal text into structured contract information.

Conclusion

The combination of **CFG + PCFG + Feature Structures + Earley Parsing** provides a more accurate and reliable approach for legal contract analysis. It reduces incorrect entity relationships, correctly

identifies obligations and deadlines, and can handle the complex sentence structures commonly found in legal documents.

4. AI-Based E-Commerce Product Search System

a) Syntactic Ambiguity Using CFG

The given query is:

“Find laptops with a touchscreen for students under ₹60,000.”

A simple CFG can be represented as:

$S \rightarrow VP$

$VP \rightarrow V\ NP\ PP\ PP\ PP$

$NP \rightarrow N$

$PP \rightarrow P\ NP$

Interpretation 1 – Intended Meaning

[Find

[laptops

[with a touchscreen]

[for students]

[under ₹60,000]]]

Meaning:

- Product = Laptop
- Feature = Touchscreen
- Target users = Students
- Price = Below ₹60,000

Interpretation 2 – Ambiguous Meaning

[Find

[laptops

[with [a touchscreen for students]]]

[under ₹60,000]]

Here, **“for students”** is attached to **“touchscreen”** rather than directly to **“laptops.”**

This is called **prepositional phrase (PP) attachment ambiguity**. A basic CFG can generate both structures but cannot determine the intended interpretation.

b) Limitations of Basic CFG

Basic CFG has several limitations when processing e-commerce queries:

1. **Ambiguous attachment:** It may attach phrases to the wrong noun or verb.

2. **Coordination:** Queries such as “*laptops and tablets with touchscreen*” can have multiple interpretations.
3. **Missing words:** Queries like “*laptop under 60k touchscreen*” are incomplete but understandable to humans.
4. **Informal language:** Users may use abbreviations such as “*60k,*” “*cheap laptop,*” or “*good laptop for college.*”
5. **Long descriptions:** Product names and descriptions may contain complex structures.
6. **No probability:** CFG cannot select the most likely interpretation among multiple parse trees.
7. **Limited semantic understanding:** CFG does not directly identify product attributes, users, or price constraints.
8. **Real-time limitations:** Processing many possible parse trees can increase response time.

Therefore, CFG alone is not sufficient for a large-scale conversational e-commerce search system.

c) Improved Parsing Architecture

The proposed architecture combines **CFG, PCFG, Feature Structures, and Earley Parsing.**

User Query



Text Preprocessing



Tokenization



Earley Parser



Possible Parse Structures



PCFG Probability Ranking



Feature Structure Analysis



Semantic Information Extraction



Search Constraints



Product Search Engine



Ranked Results

1. CFG

CFG provides the basic grammatical structure of the user's query.

2. Earley Parsing

Earley Parsing handles:

- Ambiguous queries
- Incomplete queries
- Long sentences
- Different possible parse structures

It can process a query even when the user does not provide a complete grammatical sentence.

3. PCFG

PCFG assigns probabilities to different parse structures.

For the given query, it can rank:

for students → modifies laptops

higher than:

for students → modifies touchscreen

Thus, the intended interpretation can be selected.

4. Feature Structures

Feature Structures represent the extracted product constraints:

Product = Laptop

Feature = Touchscreen

Target User = Student

Maximum Price = ₹60,000

The final structured query becomes:

Product: Laptop

Screen: Touchscreen

User: Student

Price: $\leq$ ₹60,000

d) Justification of the Proposed Solution

1. Improved Search Precision

The system correctly identifies the relationship between the product and its attributes.

For example:

Product → Laptop

Feature → Touchscreen

User → Student

Price Limit → ₹60,000

This prevents irrelevant products from being displayed.

2. Faster Response Time

Earley Parsing avoids unnecessary repeated parsing and can efficiently maintain possible interpretations. This is useful for real-time product searches.

3. Better Handling of Informal Queries

The system can handle queries such as:

“Laptop touchscreen under 60k”

“Good laptop for student”

“Touch laptop below ₹60000”

even though these are not complete grammatical sentences.

4. Better Customer Experience

Customers can use natural conversational language instead of manually selecting filters.

For example:

User:

“Find laptops with a touchscreen for students under ₹60,000.”

System understands:

Laptop

+ Touchscreen

+ Student suitable

+ Price $\leq$ ₹60,000

and returns relevant products.

5. Scalability

The architecture can process a large number of customer queries and connect the extracted constraints directly to the product database or search engine.

Comparison

Factor	Basic CFG	Proposed Approach
Ambiguity	Limited	PCFG resolves alternatives
Informal queries	Poor	Better handling
Missing words	Poor	Earley supports partial input
Product attributes	Limited	Feature Structures
Search precision	Moderate	High
Response time	Can be slow	More efficient
Scalability	Moderate	High
Customer experience	Limited	Improved

Conclusion

The combination of CFG, PCFG, Feature Structures, and Earley Parsing provides a more effective solution for conversational e-commerce search. CFG provides grammatical rules, Earley Parsing handles

complex and incomplete queries, PCFG resolves ambiguity, and Feature Structures identify product attributes and constraints. Therefore, the proposed system can provide higher search precision, faster responses, better scalability, and improved customer satisfaction.

5. Intelligent Airline Voice Assistant

a) Syntactic Structures and Ambiguities

The spoken command is:

“Change my flight to Mumbai tomorrow with two passengers.”

A simple CFG structure can be:

$S \rightarrow VP$

$VP \rightarrow V NP PP Adv PP$

$NP \rightarrow Poss N$

$PP \rightarrow P NP$

Intended Interpretation

[Change

[my flight]

[to Mumbai]

[tomorrow]

[with two passengers]]

Meaning:

- Action = Change
- Flight = My flight
- Destination = Mumbai

- Date = Tomorrow
- Passengers = 2

Ambiguous Interpretation

Another possible structure is:

[Change

[my flight]

[to [Mumbai tomorrow with two passengers]]]

Here, “with two passengers” may incorrectly be associated with the destination phrase.

The main ambiguity is prepositional phrase attachment. The system must understand that “with two passengers” represents the passenger count, not an attribute of Mumbai.

Correction Ambiguity

The passenger may say:

“Change my flight to Mumbai tomorrow... actually, make that Bangalore.”

The system should understand:

Initial Destination = Mumbai

Correction = Bangalore

Final Destination = Bangalore

It should replace Mumbai rather than creating two destinations.

b) Limitations of Top-Down Parsing

Top-down parsing starts from the start symbol and predicts grammar rules. It can cause problems in real-time voice applications.

1. Excessive backtracking:

When several interpretations are possible, the parser may repeatedly try different rules.

2. Ambiguous attachment:

It may incorrectly attach “*with two passengers*” to the destination.

3. Incomplete commands:

Users may pause or stop before completing a sentence.

4. Speech recognition errors:

Spoken input may contain incorrectly recognized words.

5. Corrections:

The parser may not easily replace previously interpreted information.

6. Delayed response:

Repeated parsing and backtracking increase processing time.

7. Conversational language:

Spoken commands may not follow complete grammatical structures.

c) Proposed Parsing Architecture

The proposed system combines Earley Parsing, PCFG, and Feature Structures.

Passenger Speech



Speech Recognition



Text Preprocessing



Earley Parsing



Partial / Alternative Parse States



PCFG Probability Ranking



Feature Structure Validation



Semantic Intent Extraction



Booking Request



Airline Reservation System



Confirmation

Earley Parsing

1. Earley Parsing can process partial and incomplete input while maintaining different possible parse states.

For example:

“Change my flight to Mumbai...”



Partial Request



“tomorrow”



Updated Request



“with two passengers”



Complete Request

This is useful for spoken commands where information arrives gradually.

2. PCFG

PCFG assigns probabilities to possible interpretations.

For example:

with two passengers → passenger information

can receive a higher probability than:

with two passengers → destination attribute

Therefore, the most likely interpretation is selected.

3. Feature Structures

Feature Structures maintain important booking information:

Action = Change

Flight = Existing Flight

Destination = Mumbai

Date = Tomorrow

Passengers = 2

They also help maintain agreement and consistency between related elements.

When the passenger says:

“Actually, make that Bangalore.”

the feature structure is updated:

Destination: Mumbai → Bangalore

d) Processing Workflow

Step 1 – Speech Input

The passenger speaks:

“Change my flight to Mumbai tomorrow with two passengers.”

Step 2 – Speech Recognition

The voice assistant converts speech into text:

Change | my | flight | to | Mumbai | tomorrow |

with | two | passengers

Step 3 – Earley Parsing

The parser identifies possible structures and maintains partial parsing states.

Step 4 – PCFG Ranking

PCFG ranks the possible interpretations and selects the most probable one.

Step 5 – Feature Structure Creation

The system creates:

Intent = Modify Flight

Destination = Mumbai

Date = Tomorrow

Passengers = 2

Step 6 – Correction Handling

If the passenger says:

“Actually, make that Bangalore.”

the system updates:

Destination = Bangalore

The previous destination Mumbai is replaced.

Step 7 – Structured Booking Request

The final request becomes:

Action = Change Flight

Destination = Bangalore

Date = Tomorrow

Passengers = 2

Step 8 – Reservation System

The structured information is sent to the airline booking system.

Step 9 – Confirmation

The assistant confirms the updated flight details with the passenger.

Advantages of the Proposed Method

Factor	Top-Down Parser	Proposed Approach
Ambiguity	Poor	PCFG resolves ambiguity
Backtracking	High	Reduced
Partial input	Limited	Earley handles it
Corrections	Difficult	Feature updates
Speech errors	Limited handling	Better contextual processing
Response time	Can be slow	Faster
Real-time use	Limited	Highly suitable

Conclusion

The combination of Earley Parsing, PCFG, and Feature Structures is well suited for a real-time airline voice assistant. Earley Parsing handles incomplete and complex spoken input, PCFG resolves ambiguous interpretations, and Feature Structures maintain consistent booking information and support corrections

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Sub. Total	Total %
1	4	4	4	4	4	20	87
2	4	3	3	4	4	18	
3	4	3	3	3	3	16	
4	3	3	3	3	4	16	
5	4	3	3	3	4	17	

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____